INTERNATIONAL UNIVERSITY

DEPARTMENT OF PHYSICS

REPORT
MIDTERM PROJECT

Lecturer: Truong Thi Ngoc Phuong

LIBRARY EQUIPMENT MANAGEMENT APP

Students 's Name:

Le Vo Anh Thu – SESEIU24024

TABLE OF CONTENT

1. PROJECT OVERVIEW.....2
1.1. Background & Academic Objectives..... 2
2. OBJECT-ORIENTED PROGRAMMING ARCHITECTURE (TASK 1: OOP DESIGN).....2
2.1. Inheritance..... 3
2.2. Encapsulation & Immutability.....3
2.3. Polymorphism & Safe Downcasting..... 4
3. USER INTERFACE ARCHITECTURE (TASK 2: UI DESIGN).....4
4. CONTROLLER & BUSINESS LOGIC (TASK 3: CONTROLLER LAYER).....6
5. VERIFICATION & TEST PROCEDURES.....7
7. CONCLUSION.....8

1. PROJECT OVERVIEW

1.1. Background & Academic Objectives

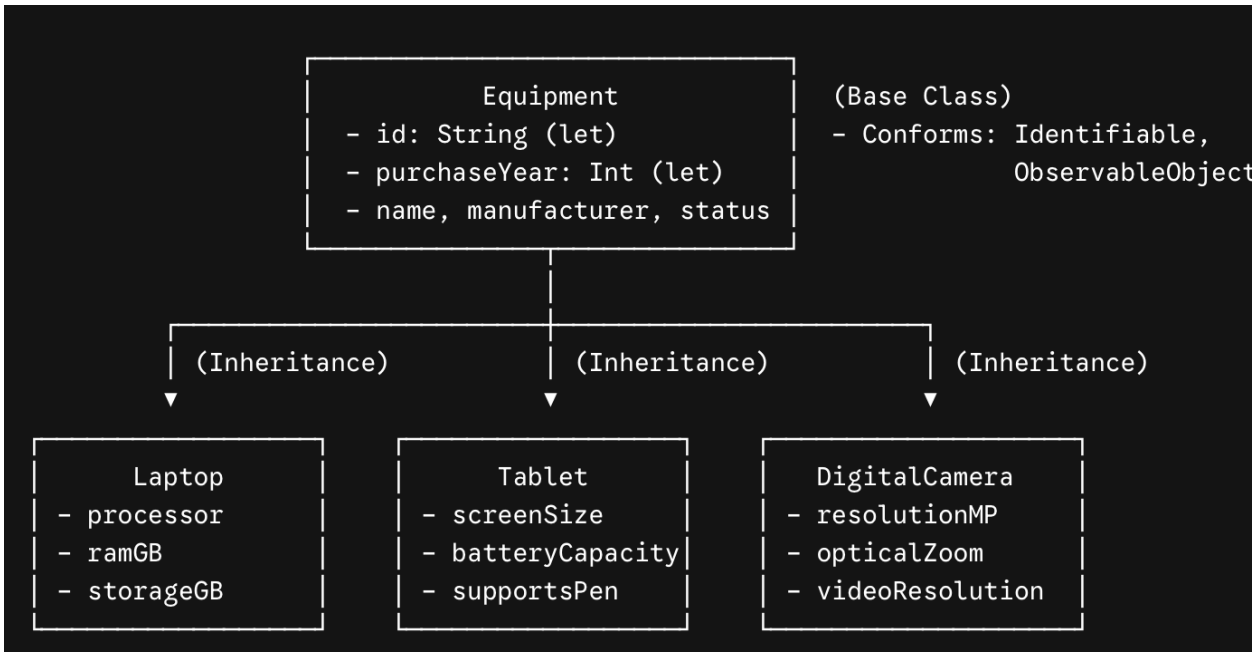
In academic and research environments, digitizing the cataloging and tracking of shared, high-value department hardware—such as laptops, tablets, and dedicated digital cameras—is essential to mitigate asset loss, ensure transaction accountability, and maximize resource utilization for research and coursework.

The **University Library Equipment Management System** is a native iOS application engineered in **Swift** utilizing the **SwiftUI** declarative framework. The technical deliverables for this midterm assessment comprise:

1. Full implementation of **Object-Oriented Programming (OOP)** principles: Inheritance, Encapsulation,Polymorphism, and Data Hiding.
2. Architectural decoupling adhering to the **MVC / MVVM** software design pattern, ensuring that user interface views remain completely isolated from underlying business validation rules.
3. A clean, responsive user experience constrained to a maximum of **three core navigation screens**, stylized with an academic Pink Palette and custom background assets (`D2`, `D3`, and `D4`).

2. OBJECT-ORIENTED PROGRAMMING ARCHITECTURE (TASK 1: OOP DESIGN)

The domain hierarchy is structured to prevent code duplication, enforce data encapsulation, and preserve model invariants:



None

2.1. Inheritance

- **Base Domain Class (`Equipment`)**: Encapsulates common attributes shared across all institutional equipment:identifier (`id`), display name (`name`), manufacturer (`manufacturer`), acquisition year (`purchaseYear`), and availability state (`status: AvailabilityStatus`).
- **Specialized Subclasses (`Laptop`, `Tablet`, `DigitalCamera`)**: Derive directly from `Equipment`, expanding domain specifications with category-specific hardware metrics through delegating initializers (`super.init(...)`).

2.2. Encapsulation & Immutability

- **Access Control (`private(set)`)**: All mutable state properties are declared using `private(set) var`. External classes and view structures cannot directly overwrite internal state fields (e.g., executing `equipment.status = .borrowed` or assigning arbitrary numeric values is blocked at compile time).

State changes must occur via guarded domain methods: `markAsBorrowed()`, `markAsReturned()`, `updateDetails()`, and `updateSpecs()`.

- **Immutability Enforcement**: The core hardware identifier (`id`) and acquisition year (`purchaseYear`) are declared as constant properties (`let`). Once an item is registered into the inventory, these values cannot be re-assigned.
- **Failable Initializers (`init?`) & Input Validation**: The base class and all derived subclasses implement failable initializers to prevent the instantiation of invalid models:
    - Empty or whitespace-only strings are rejected via `.trimmingCharacters(in: .whitespacesAndNewlines)`.
    - The `purchaseYear` must fall within the valid range: 1990<purchaseYear≤currentYear1990<purchaseYear≤currentYear.
    - Hardware specifications (RAM, Storage, Screen Size, Battery Capacity, Sensor Resolution) are strictly validated: values must satisfy spec>0spec>0.

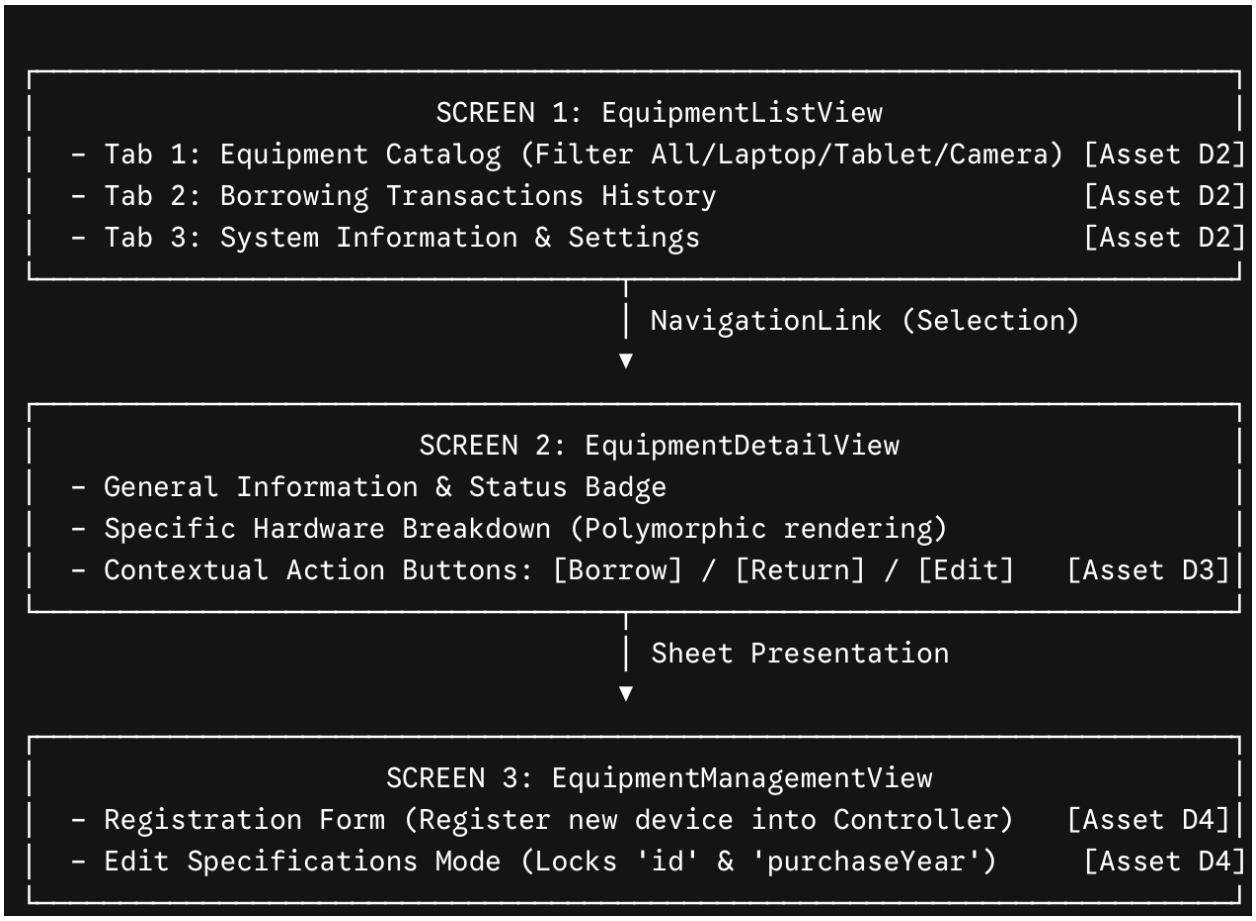
2.3. Polymorphism & Safe Downcasting

- The computed property `equipmentType` is overridden in each subclass to return the appropriate enumeration case (`.laptop`, `.tablet`, `.camera`).
- Within the presentation layer, the application utilizes safe dynamic downcasting (`if let laptop = equipment as? Laptop`, `as? Tablet`, `as? DigitalCamera`) to render specialized technical specifications dynamically without breaking the general `[Equipment]` collection abstraction.

3. USER INTERFACE ARCHITECTURE (TASK 2: UI DESIGN)

The presentation layer complies with the design constraint of three primary functional screens, incorporating custom graphic assets (`D2`, `D3`, `D4`) and high-contrast, rounded

card components:



None

1. **Screen 1 — Equipment Catalog & Overview (`EquipmentListView`)**:
    - Uses a native `TabView` structure containing three dedicated sections: Equipment Catalog (`Equipment`),Transaction History (`Borrowings`), and Application Information (`Settings`).
    - Eliminates title-overlap rendering glitches by deploying a custom navigation bar and a translucent category `Picker(.segmented)` positioned over the `"D2"` background asset.
    - Devices are rendered inside translucent list cards (`opacity: 0.88`) with color-coded status badges (`Available` in green, `On Loan` in orange).

2. **Screen 2 — Specification & Action View (`EquipmentDetailView`)**:
    - Displays general administrative metadata and polymorphic hardware specifications.
    - If an item is on loan, active borrower details and scheduled due dates are displayed in an alert card.
    - Features dynamic action buttons: displays the `Borrow` action if the item is `Available`, automatically switches to `Return` if `On Loan`, and provides an `Edit` action sheet trigger. Backed by asset `"D3"`.
3. **Screen 3 — Administration & Registration Console (`EquipmentManagementView`)**:
    - Operates under dual modes (`ManagementMode`): New Registration (`.register`) and Existing Record Update (`.edit`).
    - When operating in `.edit` mode, the critical identifier (`id`) and acquisition year (`purchaseYear`) form fields are rendered read-only via `.disabled(true)`, preserving database integrity. Backed by asset `"D4"`.

4. CONTROLLER & BUSINESS LOGIC (TASK 3: CONTROLLER LAYER)

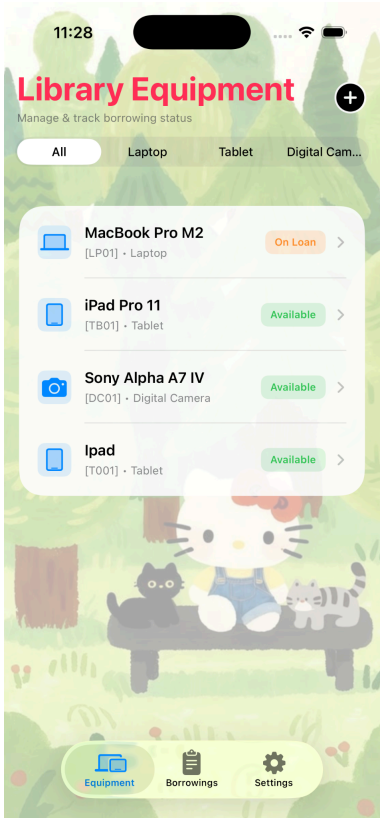
The `LibraryEquipmentController` functions as the centralized ViewModel/Controller managing shared mutable state,conforming to `ObservableObject` and driving reactive updates across views:

- **State Collections**: Manages the published collections `@Published var inventory: [Equipment]` and `@Published var transactions: [BorrowTransaction]`.
- **Identifier Uniqueness (`registerEquipment`)**: Validates that incoming registration requests do not duplicate an existing `id` (case-insensitive evaluation).
- **Borrowing Transaction Validation (`borrowEquipment`)**:
    - Validates that the borrower name is non-empty.
    - Enforces that the expected return timestamp is in the future.

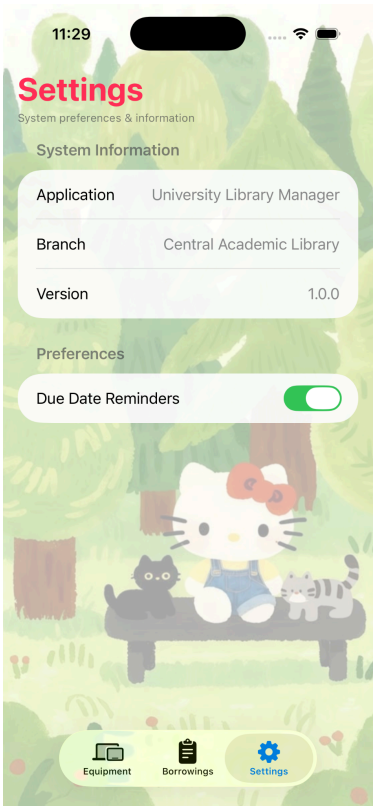
    - Verifies that `equipment.status == .available`. If the device is currently on loan, the transaction is rejected, preventing concurrent double-borrowing.
    - Upon validation, transitions the status via `equipment.markAsBorrowed()`, appends an active `BorrowTransaction`, and publishes state changes via `objectWillChange.send()`.
- **Return Transaction Validation (`returnEquipment`)**:
    - Verifies that `equipment.status == .borrowed`. Rejects attempts to return items that are already marked available.
    - Closes the active loan record by recording `actualReturnDate = Date()`, resets device status to `available`,and broadcasts the update to subscribed views.

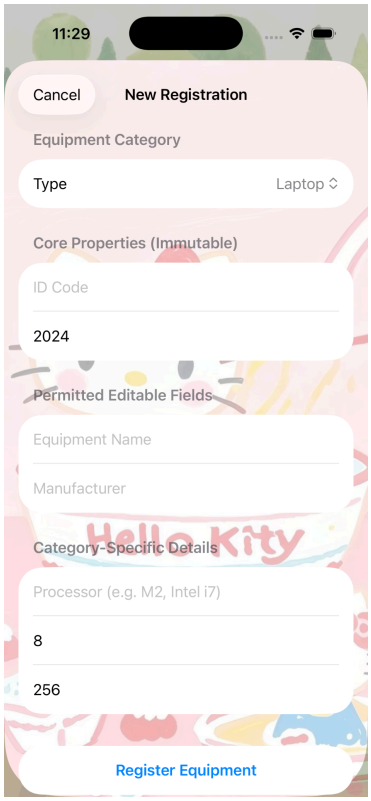
5. VERIFICATION & TEST PROCEDURES

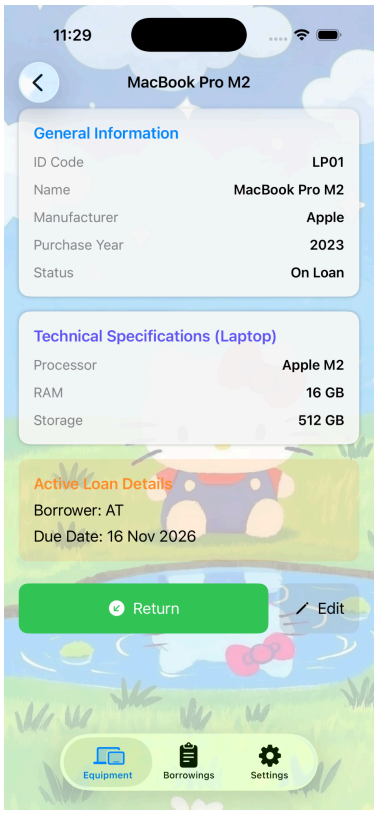
The core functional requirements and edge cases were tested on an iPhone 17 Pro iOS Simulator:

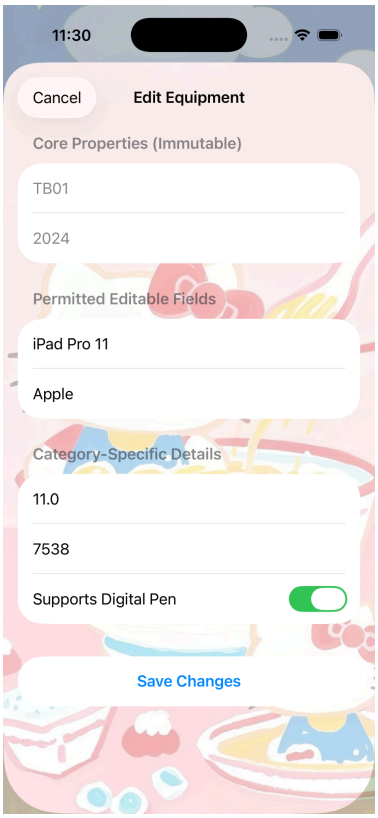

11:28
Library Equipment
Manage & track borrowing status
All
Laptop
Tablet
Digital Cam...
MacBook Pro M2
[LP01] · Laptop
On Loan
iPad Pro 11
[TB01] · Tablet
Available
Sony Alpha A7 IV
[DC01] · Digital Camera
Available
Ipad
[T001] · Tablet
Available
Equipment
Borrowings
Settings


11:28
Borrowings
Active and past loan records
ID: LP01
Active Loan
Borrower: AT
Date: 9 Nov 2026 → Due: 16 Nov 2026
Equipment
Borrowings
Settings


11:29
Settings
System preferences & information
System Information
Application
University Library Manager
Branch
Central Academic Library
Version
1.0.0
Preferences
Due Date Reminders
Equipment
Borrowings
Settings


11:29
Cancel
New Registration
Equipment Category
Type
Laptop
Core Properties (Immutable)
ID Code
2024
Permitted Editable Fields
Equipment Name
Manufacturer
Category-Specific Details
Processor (e.g. M2, Intel i7)
8
256
Register Equipment


11:29
MacBook Pro M2
General Information
ID Code
LP01
Name
MacBook Pro M2
Manufacturer
Apple
Purchase Year
2023
Status
On Loan
Technical Specifications (Laptop)
Processor
Apple M2
RAM
16 GB
Storage
512 GB
Active Loan Details
Borrower: AT
Due Date: 16 Nov 2026
Return
Edit
Equipment
Borrowings
Settings


11:30
Cancel
Edit Equipment
Core Properties (Immutable)
TB01
2024
Permitted Editable Fields
iPad Pro 11
Apple
Category-Specific Details
11.0
7538
Supports Digital Pen
Save Changes

6. VERSION CONTROL & DEPLOYMENT METRICS

Project progress and source code versions are managed via Git and hosted on GitHub:

- **Repository Owner:** `leanhthu1026-cpu` (leanhthu1026@gmail.com)
  Screenshot 2026-11-09 at 11.12.09.png
- **Repository Name:** `MIDTERM`
  Screenshot 2026-11-09 at 11.12.09.png
- **Target Branch:** Single unified `main` branch containing all verified source files and configuration manifests.
  Screenshot 2026-11-09 at 11.12.09.png+ 1
- **Commit History:** Demonstrates iterative refactoring from generic template views to fully implemented,production-ready OOP models and SwiftUI controllers (evidenced by commit `MIDTERM UPLOAD` – `7350eab`).
- **Build Integrity:** Compiles cleanly on Xcode 15+ / Xcode 16+ targeting iOS 17.0+ with **0 Warnings, 0 Errors (Build Succeeded)**.

7. CONCLUSION

The **University Library Equipment Management System** satisfies all academic criteria stipulated for the assessment:

1. Implements an object-oriented domain hierarchy leveraging inheritance, strict `private(set)` encapsulation, and input validation.
2. Delivers an MVC/MVVM SwiftUI interface bounded within three functional screens, maintaining aesthetic consistency via custom image assets (`D2`, `D3`, `D4`).
3. Enforces data invariants, unique identification, and transactional validity within a decoupled controller layer.